ACE Cyber Testbed Setup & Configuration

Purpose:
This document provides a structured record of how the research testbed environment was designed, configured, and validated to support your project's experiments. It ensures your environment is **reproducible**, **secure**, and **aligned with research goals**.

---

1. Project Context

Field	Response
Student Name(s)	Teaundra Thompson
Research Title	IDS Resiliency Against Evasion Attacks
Faculty Advisor	Professor Henderson
DoD/ACE Mentor	Professor Littleton
Version	v1.8
Date	12/9/2025

---

2. Testbed Purpose and Scope

Describe the **overall purpose** of the testbed. Explain how it supports your research question and hypothesis.

Include:

- What is the **goal** of this testbed? (e.g., simulate SIEM alert handling, test anomaly detection, etc.)
- What research **variables** or conditions will be studied here?
- What **outcomes** do you expect from experiments in this environment?
- What **limitations** does this environment intentionally impose (e.g., isolated network, simulated data)?

Example: The testbed emulates a CSSP SOC environment using Elastic Stack components to generate and analyze simulated network traffic for false positive reduction experiments.

ACE Cyber Testbed Setup & Configuration

The testbed will emulate a MLP ML-based IDS under adversarial evasion attacks. The baseline detection accuracy, and true/false positive rate will be studied to determine the resiliency of the system. I expect my hypothesis to be proved true due to the meticulous control of the variables in this experiment. The testbed environment will impose limitations on live network monitoring; I will only feed pcap files into the IDS. Upon feeding a pcap file into Zeek, it will generate logs and send it to the MLP model and the MLP will create a log identifying any malicious entries and send that log to Zeek.

---

3. Testbed Architecture Overview

Provide a **high-level diagram** or description of your testbed topology.

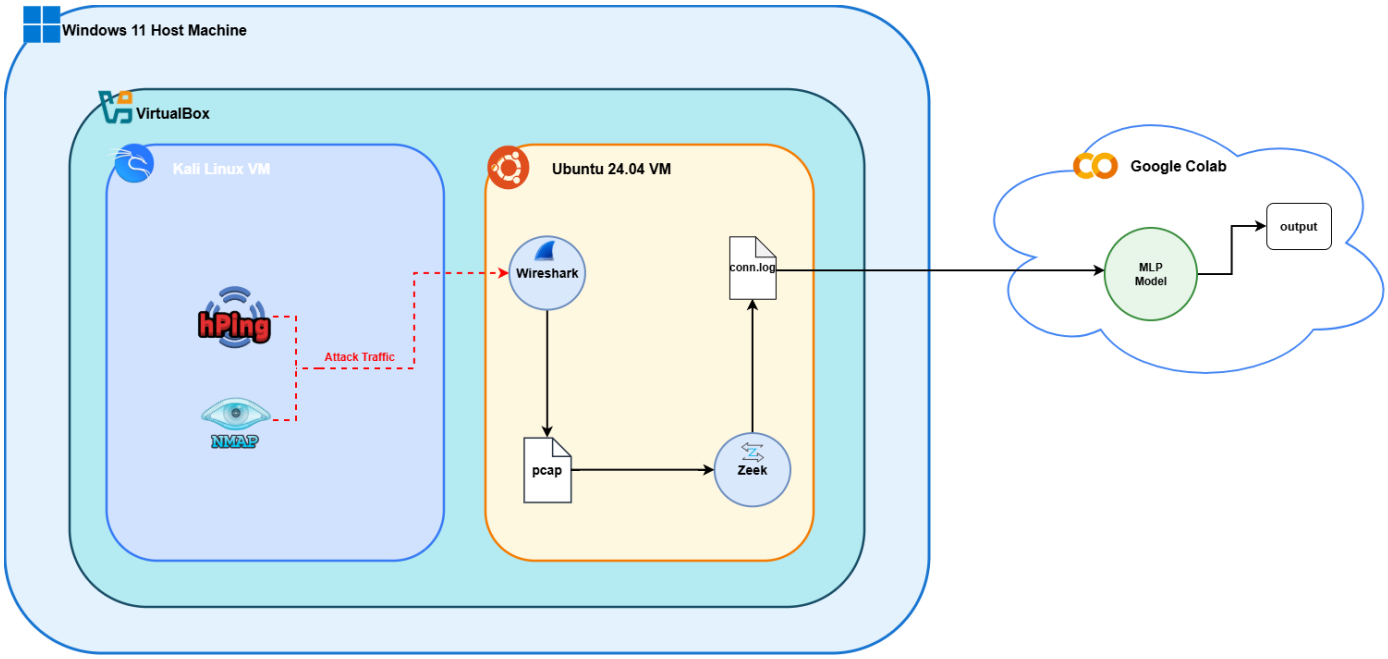
Include:

- All **nodes, VMs, or containers** and their roles
- Network segmentation and connectivity (firewalls, VLANs, bridges)
- Any **virtualization platforms** (VMware, VirtualBox, KVM, etc.)
- Any **container orchestration tools** (Docker, Podman, Kubernetes)

Component	Function	Host Type	OS / Version	Network Role
Ubuntu/Zeek VM	Host baseline IDS	VM	Ubuntu 24.04.3	NAT and Host-Only Adapter
Kali Linux	Attacker VM	VM	2025.2	Host-Only Adapter
VirtualBox	Hosting IDS model	Hypervisor	7.1.6	NAT
Google Colab	Create, train, and host MLP model	Cloud	N/A	None (conn.logs will be uploaded to environment)

(Replace sample data with your own.)

Attach a **diagram(s)** (Visio, draw.io, or screenshot) showing relationships between components (topology, dataflows, etc.).



---

4. Hardware and Virtual Resources

List the **hardware and compute resources** supporting your testbed.

Resource	Description / Role	Specs	Allocation
Ubuntu/Zeek VM	ML-Based IDS/Hosted on Windows machine	N/A	19.7 GB RAM, 8vCPU, 200 GB storage
Kali Linux VM	Attacker VM, simulate malicious traffic to Zeek VM	N/A	2 GB RAM, 2vCPU, 80.09 GB storage
Windows 11 Host Machine	Hosting machine for VirtualBox + VMs	32 GB RAM (speed 3200 MT/s), 1.38 TB storage, 8 GB GPU	N/A
Google Colab	Train, test and host MLP model	12.7 GB RAM, 225.8 GB storage	N/A

Include:

- CPU, RAM, and disk size per node
- Host operating system
- Virtualization platform (e.g., ESXi, VirtualBox)

---

5. Software Stack and Tools

Provide details on all software tools installed.

Component	Version	Purpose	Installation Method
Zeek	8.0.1	Base IDS	Terminal
Scapy	2.5.0	Pcap manipulation	Terminal
libpcap	1.10.4	To support scapy	Terminal
Wireshark	4.2.2	Create pcap files	Terminal
Hping3	3.0.0	Simulate DoS SYN flood	Terminal

Include any **scripts, frameworks, or libraries** (e.g., TensorFlow, Scikit-learn).

Notes: Within the Colab environment TensorFlow, Scikit-learn, pandas, numpy were used within the python code for the MLP.

---

6. Network Configuration

Document **network setup and addressing scheme**.

Network Name	Subnet	VLAN / Segment	Purpose	Notes
Testbed Internet	10.0.2.0/24	N/A	Download files, software, and access Colab	NAT Adapter
Internal Network	192.168.56.0/24	N/A	Capture internal traffic from Kali Linux VM	Host-Only Adapter
Google Colab	N/A	Cloud	Host MLP model for training and log analysis	Not directly connected to testbed environment

Include:

- **Firewall rules** or **routing tables** (if applicable)
- **NAT**, **bridging**, or **port forwarding** configurations
- How isolation or segmentation prevents unintended exposure

---

7. Security Controls

Summarize **security measures** implemented.

Include:

- Account and access control setup
- Use of encryption (TLS, SSH, VPN)
- Network isolation or sandboxing
- Logging/auditing measures
- Vulnerability or compliance scans (if applicable)

Example: All VMs run with non-root users; SSH access restricted to specific keys. Testbed is isolated from enterprise network via host-only adapters.

The Zeek VM will be connected to the internet via a NAT adapter to assist with downloading necessary software; it will also have a secondary Host-only network adapter to capture internal traffic from the Kali Linux VM. The Kali Linux VM will have only one adapter, the Host-Only adapter to prevent malicious traffic from leaking out into the external network. The feature for live network monitoring in Zeek will be disabled to prevent exposure of sensitive data. During the training phase of the MLP machine learning model in Google Colab, it will be trained on the UFW-ZeekData24 dataset which is publicly available.

---

8. Test Data Integration

Describe how data sources are integrated into the testbed.

Include:

- Data ingestion tools (e.g., Filebeat, Kafka, Logstash)
- Storage paths or indices (e.g., /var/log/testdata.log, Elastic index)
- How synthetic or real datasets are loaded
- Validation steps confirming data flow

Example: Synthetic alert data from Python scripts is streamed into Elastic via Filebeat for visualization in Kibana dashboards.

For the training portion of the MLP, the UFW-ZeekData24 dataset six CSV files will be uploaded to the Colab environment from the VM's shared folder with the host machine. When the model is trained and tested, the model weights will be saved in a keras file and the preprocessors will be saved to .pkl files. Those files can be downloaded to the VM and stored in path /Desktop/testCases/MLP Model and then reuploaded to Colab whenever a new runtime is created.

For the testing portion, pcap files will be created on the VM and stored in file path " Desktop/TC…". Pcaps will be stored in their respective "TC…" folder with the corresponding test case number, such as "TC1". The pcap files will then be fed into Zeek, and Zeek will generate the conn.logs into the respective test case folder. The respective conn.log will then be uploaded to the Colab notebook environment to be analyzed by the MLP. A screenshot will be taken of the MLP's analysis and stored in the respective test case folder.

---

9. Configuration Management

Document how configurations are stored and tracked.

Include:

- Configuration tools used (e.g., Ansible, Terraform, Git)
- Repository path or Git commit hash
- Backup strategy for configurations and scripts

Config Area	Tool	Location	Versioning Method
VM Templates	VirtualBox	D:\VirtualBoxVMs\Zeek VM\Snapshots	Snapshot
Google Colab	GitHub	ladyTootie/ACE-R-D	Push to Github
(Alternate method) Google Colab	Save and pin revision	Colab	Able to see previous versions and revisions using this built-in tool

---

10. Experiment Automation (if applicable)

Describe how experiments are executed or repeated automatically.

Include:

- Any scheduling tools (cron, Airflow, Jenkins)
- Parameters or triggers used
- Output data path and logging method

Example: Cron jobs execute data replay every 15 minutes. Each run logs system metrics and alert counts to a JSON file.

The experiments will be conducted manually due to the nature of changing file paths to complete test cases. The method of manually conducting the experiments takes little time, thus there is no pertinent need for automation.

---

11. Validation and Testing

Provide evidence that the testbed functions as intended.

Include:

- Tests run to verify data flow or alert generation
- Screenshots of dashboards or log outputs
- Example command outputs
- Observations and test results

Example: Verified data pipeline end-to-end: Log source → Filebeat → Elastic → Kibana visualization. Confirmed expected alert volumes.

conn.log	46.8 kB	13 Oct 2025
dns.log	73.9 kB	13 Oct 2025
files.log	17.1 kB	13 Oct 2025
ftp.log	1.5 kB	13 Oct 2025
http.log	4.7 kB	13 Oct 2025
NoobsKeylogger.pcap	996.8 kB	13 Oct 2025
ocsp.log	12.1 kB	13 Oct 2025
packet_filter.log	90 bytes	13 Oct 2025
ssl.log	25.6 kB	13 Oct 2025
x509.log	25.0 kB	13 Oct 2025

Log output from pcap file

To feed a pcap file into Zeek and generate logs I change the directory to the respective test case folder and run command **zeek -r filename.pcapng**. Running that command does not produce output into the terminal, but it generates logs into the folder where the pcap file resides. Then the connection log is uploaded to Google Colab, and an output is produced depending on the traffic type present in the log.

12. Limitations and Known Issues

Identify any **constraints** or **bugs** encountered.

Include:

- Performance limits (CPU bottlenecks, storage caps)
- Missing dependencies
- Incomplete features or manual workarounds

Example: Replay script occasionally dropped packets under high volume due to CPU contention.

During my installation the Zeek repository's mirror metadata and packages were out of sync with what was downloading on the vm. I found a workaround where I could proceed with the installation by telling "apt" to relax its security checks. Then I told apt to re-download missing or mismatched files and proceed with the installation. There was no impact on my testbed environment from using the workaround. Checksum for Zeek install: 8741dc8f8f29d1bd4e6840e0d0022b2da609bffbb173a8b4f2b2eff3310c8c5c.

---

13. Backup and Recovery Plan

Describe how the environment is backed up and restored.

Include:

- Backup frequency and tools (snapshot, rsync, Git commit)
- Recovery procedures
- Test of restore process

  Since the testbed environment is in a VM, I will take a snapshot of the VM at least twice a week. Previous snapshots will be merged into the new snapshot to prevent excessive use of storage. To ensure a snapshot is accurate I can revert back to the snapshot and check that the IDS is intact. For the Colab environment, I have the option of uploading the notebook to GitHub or using the built-in "revision history" feature since it is a cloud environment. Colab automatically saves the notebook after every change is made, So I will utilize that feature.

---

15. Mentor Review / Approval

ACE Cyber Testbed Setup & Configuration

Reviewer	Role	Date	Notes
Faculty Advisor			
DoD/ACE Mentor			

Instructions

This document must capture **how your research environment was built** so that anyone could recreate it from scratch. It should be clear, factual, and supported by screenshots or evidence where possible.

Section-by-Section Tips:

Section	What to Include	Why It Matters
1. Project Context	Identify project and authors	Traceability
2. Purpose & Scope	Link testbed to hypothesis	Research alignment
3. Architecture	Diagrams, nodes, topology	Transparency
4. Hardware	Specs and allocation	Resource reproducibility
5. Software	Versions, tools, install methods	Environment control
6. Network	IPs, VLANs, isolation	Security and reproducibility
7. Security Controls	Access, encryption, logging	Compliance
8. Data Integration	Flow from source to analysis	Trace data movement
9. Config Mgmt	Repositories, backups	Version control
10. Automation	Scheduled tests, scripts	Reproducibility
11. Validation	Screenshots, results	Functionality proof
12. Limitations	Issues and constraints	Academic integrity

ACE Cyber Testbed Setup & Configuration

Section	What to Include	Why It Matters
13. Backup	Recovery plan	Resilience
14. Change Log	Updates history	Accountability
15. Review	Mentor sign-off	Oversight

Formatting and Submission:

- Make sure you use **consistent terminology** (VM, container, node).
- Include **screenshots** of dashboards, terminal outputs, or network diagrams.
- Reference **code or configuration files** by name and location.
- Submit as **.docx or .pdf**, with diagrams embedded or appended.
- Confirm **mentor review** before final submission.